GNU Backgammon Android Port

clavierhaus.at

Juni 2026

Contents

1. GNU Backgammon Android Port — MASTER v0.9.1

GNU Backgammon by clavierhaus.at
clavierhaus.at · Vienna · Austria
June 2026

1.1 1. Introduction & Project Objective

The objective of this project is to produce a **true port** of GNU Backgammon to the Android platform — not a stripped-down evaluator, but a complete port that mirrors 100% of the functionality of the PC version. This includes full game management, SGF import/export, analysis, rollout, cube decisions, tutor mode, and the complete match state machine.

The primary constraint is the monolithic nature of the original gnubg desktop codebase, which is tightly coupled with GTK/GNOME UI components, file I/O, global configuration state, and platform-specific threading primitives. The approach is to compile the genuine gnubg engine source — every file that is portable — and replace only what is inherently GTK-specific with Android equivalents via a clean architectural boundary: `android-app.c`.

The key architectural insight: gnubg's codebase separates cleanly into three layers:

1. **Mathematical engine** (`eval.c`, `rollout.c`, `dice.c`, `bearoff.c` etc.) — pure computation, zero UI dependencies, 100% portable
2. **Game logic layer** (`play.c`, `analysis.c`, `sgf.c`, `set.c`, `format.c` etc.) — all GTK references are behind `#if defined(USE_GTK)` guards, portable when compiled without `USE_GTK`
3. **GTK application shell** (`gnubg.c`, `progress.c`, `gtk/*.c`) — not portable, replaced by `android-app.c`

Every stub that silently drops functionality is a hole in the port. The goal is to minimise stubs to only what is genuinely GTK-specific rendering code.

1.1.1 1.1 Scope

This document covers the complete build history, all architectural decisions, every obstacle encountered and its resolution, and the exact current state of the project as of Version 6. It is intended as a working reference for any software engineer picking up or contributing to this project.

1.1.2 1.2 App Identity

Property	Value
App name	GNU Backgammon by clavierhaus.at
Android package	`com.clavierhaus.gnubg`
Minimum Android API	31 (Android 12.0, Snow Cone)
Target ABI	`arm64-v8a`
Engine source	GNU Backgammon upstream (gnubg.org)
Repository	https://github.com/clavierhaus/gnubg-android

1.2 2. Narrative of Progress

1.2.1 Phase 1: Monolithic Analysis & Prototype

Initial efforts focused on analysing the upstream GNU Backgammon source. Attempts to build the full desktop environment proved unsustainable due to deep dependencies on GTK, GLib, and global UI state variables (`ms`, `ap`, `positions`). A "Frankenstein" build approach — attempting to stub these dependencies incrementally — established that the mathematical evaluation logic was sound in isolation, but produced a fragile, non-portable build environment that could not be taken further.

1.2.2 Phase 2: Redesign — Deterministic Engineering

Phase 2 pivoted to a "Clean-Room" architecture. The attempt to port the desktop environment was discarded entirely. The "Core Engine" was isolated: a strict build hierarchy was established in which the upstream source acts as a read-only reference, and the `engine-core/` directory contains only the verified, essential mathematical components.

This phase established the orchestration scripts (`pull_dependencies.sh`, `patch_engine.sh`, `build_all.sh`) and the Patch Registry: surgical, idempotent fixes for POSIX/Android compliance.

1.2.3 Phase 3: The Pristine Headless Build (Completed)

Phase 3 achieved a fully hermetic host build of the mathematical core, confirming that the engine compiles and links cleanly on Linux when detached from the GTK/GNOME infrastructure.

When cross-compilation for Android NDK was attempted, the shim approach collapsed. The root cause: gnubg uses GLib for threading primitives (`GMutex`, `GCond`, `GThread`), dynamic data structures (`GList`, `GArray`, `GHashTable`), and character encoding conversion. Shimming these by hand created an infinite chain of type collisions. This is not a sustainable path; it is an attempt to reconstruct the Linux runtime inside the Android NDK.

1.2.4 Phase 4: GLib Cross-Compilation for Android (Completed)

The architectural decision was made to cross-compile a minimal but real GLib 2.88.1 for `arm64-v8a` Android API 28, rather than shimming GLib. This provides the full, tested GLib API that gnubg requires, with no behavioral deviations from upstream.

The GLib source was obtained from the Fedora 44 SRPM (`glib2-2.88.1-1.fc44`), ensuring Fedora's tested patches were applied. The result is `libglib-2.0.so` verified as ELF 64-bit ARM aarch64 for Android 28.

1.2.5 Phase 5: JNI Bridge & libgnubg-engine.so (Completed)

Phase 5 constructed the JNI bridge layer (`native-lib.c`, `stubs.c`, `Engine.kt`) and CMake build pipeline producing `libgnubg-engine.so`, verified as ELF 64-bit ARM aarch64 for Android 28.

1.2.6 Phase 6: NEON Acceleration & Device Verification (Completed)

Phase 6 enabled ARM NEON SIMD acceleration and verified the engine running correctly on a Pixel 8 Pro (aarch64, Android 16) via adb test harness. Two critical bugs were discovered and fixed during device testing; see §5.5 and §5.6.

Verified results on device (opening position, 1-ply cubeless):

Output	Value	Expected
`ClassifyPosition`	10 (CLASS_CONTACT)	correct
Win probability	0.5269	~0.50
Win gammon	0.1478	~0.12
Win backgammon	0.0085	~0.01
Lose gammon	0.1285	~0.12
Lose backgammon	0.0049	~0.01
`FindBestMove` 3-1	8/5 6/5	correct (textbook)

1.2.7 Phase 7: Cube Decisions & Rollout (Completed)

Phase 7 added cube decision analysis and synchronous rollout to the JNI API, both verified on device.

Cube decision (`Engine.cubeDecision()`): wraps `GeneralCubeDecisionENoLocking` + `FindCubeDecision`. Returns the full `aarOutput[2][7]` equity matrix and the `cubedecision` enum value. Verified: opening position returns `NODOUBLE_TAKE` (correct for a money game with a centred cube).

Rollout (`Engine.rollout()`): Phase 7 introduced a synchronous single-threaded rollout via `gnubg_rollout()` calling `BasicCubefulRolloutNoLocking` directly. Phase 9 supersedes this with a full multi-threaded implementation; see Phase 9. See §5.7 and §6.2.

1.2.8 Phase 8: Full Game Logic Layer (Completed)

Phase 8 is the pivotal phase — the port moves from "evaluator with stubs" to "genuine gnubg port." The game logic layer (`play.c`, `analysis.c`, `sgf.c` and all their dependencies) is compiled into `libgnubg-engine.so`.

The android-app.c architecture: Rather than adding `gnubg.c` (the GTK application shell, ~8000 lines, deeply GTK-dependent), a new file `jni-bridge/src/android-app.c` was created as the Android equivalent. It contains: - Functions extracted verbatim from `gnubg.c`: `InitBoard`, `NextToken`, `NextTokenGeneral`, `DisectPath`, `ParseNumber`, `ParseReal`, `ParsePosition`, `ParseKeyValue`, `ParsePlayer`, `SetToggle`, `CompareNames`, `GetMatchStateCubeInfo`, `find_skills`, `swapGame`, `NameIsKey`, `AddKeyName`, `DeleteKeyName`, `CommandSwapPlayers`, `SmartSit`, `asyncEvalRoll`, `asyncMoveDecisionE`, `asyncFindMove`, `asyncCubeDecision`, `asyncAnalyzeMove`, `GetEvalChequer`, `GetEvalCube`, `GetEvalMoveFilter` - Android replacements for GTK functions: `playSound→gnubg_on_sound_even` `ShowBoard→gnubg_on_board_changed`, `UpdateSetting→gnubg_on_setting_changed`, `CommandFirstGame/Move→gn` `GiveAdvice→gnubg_on_navigation_event`, `RunAsyncProcess` (synchronous), `get_input_discard` (returns TRUE), `confirmOverwrite` (returns TRUE) - `RolloutProgressStart/Progress/End` replacing `progress.c` (deeply GTK) - Sound infrastructure replacing `sound.c` - All globals previously in `gnubg.c`

Source files added to build: `play.c`, `analysis.c`, `format.c`, `formatgs.c`, `drawboard.c`, `external.c`, `glib-ext.c`, `matchid.c`, `set.c`, `renderprefs.c`, `sgf.c`, `sgf_y.c` (bison-generated), `sgf_l.c` (flex-generated)

SGF parser generation: `sgf_y.y` and `sgf_l.l` are yacc/lex source files. They must be processed before building:

```
cd engine-core
bison -d -o sgf_y.c sgf_y.y
flex -o lex.yy.c sgf_l.l && mv lex.yy.c sgf_l.c
sed -i '1s/^/#include <unistd.h>\n/' sgf_l.c
```

Verified: All 6 existing test harness tests still pass on Pixel 8 Pro after adding the full game logic layer.

1.2.9 Phase 9: Multi-threaded Rollout & SGF JNI API (Completed)

Phase 9 re-enables `USE_MULTITHREAD` and implements production-quality parallel rollout, then adds the SGF load/save JNI entry points to complete the engine feature set.

Multi-threaded rollout: A persistent `GThreadPool` is created at `gnubg_init_rollout()` with `g_get_num_processors()` threads. Each rollout trial is a separate task dispatched via `g_thread_pool_push()`. A `GMutex`/`GCond` barrier blocks the calling thread until all trials complete. Per-thread state is managed via two `GPrivate` keys: `gnubg_tls_key` (owns `ThreadLocalData` with `aMoves` and `pnnState` buffers) and `gnubg_rng_key` (owns an isolated `rngcontext` copy, preventing RNG data races). Result structs are 64-byte cache-line aligned to prevent false sharing on aarch64. The scatter-gather merge computes mean and standard deviation after all workers finish.

SGF JNI entry points: `Engine.loadSGF(path)` calls `CommandLoadMatch()` which handles `FreeMatch`, `ClearMatch`, `RestoreGame`, and `UpdateSettings` — the complete match load sequence as implemented in `sgf.c`. `Engine.saveSGF(path)` calls `CommandSaveMatch()` which handles the `SaveGame` loop, file I/O, `setDefaultFileName`, and `delete_autosave`. Both entry points are thread-safe via `gnubg_lock`.

Engine feature-complete: After Phase 9, `libgnubg-engine.so` provides 9 JNI entry points covering the full gnubg evaluation and game management feature set. See §6.2 for the complete API reference.

1.3 3. Architecture

1.3.1 3.1 Directory Structure

All project artefacts reside under `/home/erweitert/gnubg-android/`:

```
/home/erweitert/gnubg-android/
  android-sdk/                <- Android SDK & NDK (NDK 27.0.11718014)
  engine-core/                <- gnubg source (config.h is Android-patched)
    lib/                      <- gnubg math library (neuralnet, SFMT, cache...)
    config.h                  <- PATCHED: host-generated flags removed
    lib/config.h              <- Wrapper: #include "../config.h"
  upstream-source/gnubg/      <- Unmodified upstream reference
  glib-2.88.1/                <- GLib source (Fedora SRPM, patched)
  glib-android-build/         <- Meson build directory (out-of-tree)
  jni-bridge/                 <- JNI bridge layer
    CMakeLists.txt            <- NDK CMake build
    config.h                  <- Shadow config.h for engine-core/*.c
    external/glib/            <- Installed GLib (headers + .so files)
    src/native-lib.c          <- JNI entry points
    src/stubs.c               <- UI/threading layer stubs + TLD init
    src/com/clavierhaus/gnubg/Engine.kt  <- Kotlin JNI declarations
    build/libgnubg-engine.so <- Final deliverable
  test-harness/               <- Android adb test harness
    CMakeLists.txt            <- Builds standalone Android executable
    src/main.c                <- Test entry point
    src/stubs.c               <- Same stubs as jni-bridge
  android-arm64.cross         <- Meson cross-file for GLib build
  build_glib_android.sh       <- GLib cross-build script
  doc/                        <- Documentation
```

```
Makefile                 <- make / make pdf / make tex
gnubg.tex                <- XeLaTeX template
```

1.3.2 3.2 Build Toolchain

Component	Version / Detail
Host OS	Fedora 44, KDE Plasma 6/Wayland
Compiler (host)	GCC 16.1.1 (Red Hat)
Compiler (Android)	Clang 18.0.1 (NDK r27-beta1, build 11718014)
NDK	27.0.11718014 at `android-sdk/ndk/27.0.11718014/`
Meson	1.11.1 (Fedora package)
Ninja	1.13.2 (Fedora package)
CMake	4.3.0 (Fedora package)
GLib	2.88.1 (Fedora SRPM `glib2-2.88.1-1.fc44`)
Target ABI	`arm64-v8a` (`aarch64-linux-android28`)
Android API level	28 (Android 9.0 Pie)
Test device	Pixel 8 Pro (aarch64, Android 16)

1.4 4. GLib Cross-Compilation for Android

1.4.1 4.1 Rationale

gnubg's engine uses GLib for: threading primitives (`GMutex`, `GCond`, `GThread`); dynamic data structures (`GList`, `GArray`, `GHashTable`); character encoding conversion (iconv wrapper); memory allocation wrappers; assertion macros; and logging. The engine cannot function without these. Shimming them by hand is not viable: each shim requires further GLib internals, leading to infinite type collision regression.

Cross-compiling real GLib for Android provides the full, tested implementation that gnubg was written against, with no behavioral deviation. The investment is a one-time build; the resulting `.so` files are reused for all future engine builds.

1.4.2 4.2 Meson Cross-File (android-arm64.cross)

The cross-file specifies the NDK toolchain binaries, target machine description, and platform properties that Meson cannot auto-detect in a cross build. Critical design points:

- **Do NOT define `__ANDROID_API__` in `c_args`:** The compiler binary `aarch64-linux-android28-clang` already encodes API level 28. NDK clang 18 defines it internally; a redefinition causes `-Werror` failures in Meson's size detection probes.
- **Explicit `sizeof_*` properties:** Provided to bypass Meson's cross-detection probes, which fail because they attempt to run target binaries on the host.
- **`pkg-config` disabled:** Set to `false` to prevent Meson from accidentally finding host-side `.pc` files.

1.4.3 4.3 Key Build Obstacles & Resolutions

Obstacle	Resolution
Unknown option `iconv`	GLib 2.88 removed `-Diconv` as a Meson option. iconv detection is now via `dependency('iconv')` internally.
`size_t` detection failure	Caused by `-D__ANDROID_API__=26` in `c_args` conflicting with the compiler's built-in definition. Removed the redundant define.
iconv not found (API 26)	Bionic declares iconv under `#if __ANDROID_API__ >= 28`. Bumped target from API 26 to API 28, which is the correct minimum for this project.
`dependency('iconv')` fails on Android	GLib's Meson uses `clang++` to probe `iconv_open()`, but Bionic's `iconv.h` lacks `extern "C"` linkage for C++ probing. Patched GLib's `meson.build` to use `declare_dependency()` on Android since iconv is in Bionic libc with no separate library.
`libintl.h` not found	GLib's `gi18n.h` requires `libintl.h`. The GLib build's `proxy-libintl` subproject provides both the header and `libintl.so`. Added `external/glib/include` to the CMake include path.

1.4.4 4.4 Installed Artefacts

The GLib build installs to `jni-bridge/external/glib/` and provides:

- `lib/libglib-2.0.so` — core GLib (the primary dependency)
- `lib/libintl.so` — proxy-libintl stub (required by `gi18n.h` / gettext macros)
- `lib/libffi.so` — libffi (GLib closure support)
- `lib/libgio-2.0.so`, `libgobject-2.0.so`, `libgmodule-2.0.so`, `libgthread-2.0.so` — GLib sublibraries
- `include/glib-2.0/` — GLib headers
- `include/libintl.h` — proxy-libintl header
- `lib/glib-2.0/include/glibconfig.h` — platform-specific GLib configuration

1.5 5. JNI Bridge Construction

1.5.1 5.1 Source File Inventory

File	Purpose
`jni-bridge/CMakeLists.txt`	NDK CMake build definition. Lists all compiled source files, include paths, compile definitions, and linked libraries.
`jni-bridge/config.h`	Shadow `config.h` for `engine-core/*.c` files. See §5.3.

File	Purpose
`jni-bridge/format.h`	Empty stub header — `rollout.c` includes it but uses no symbols. Prevents GTK chain.
`jni-bridge/matchid.h`	Empty stub header — same rationale as `format.h`.
`jni-bridge/analysis.h`	Minimal stub — `sgf.c` includes it; `RestoreDoubleAnalysis/RestoreMoveAnalysis` are defined within `sgf.c` itself.
`jni-bridge/drawboard.h`	Stub defining `FORMATEDMOVESIZE 29`. Prevents GTK rendering chain.
`jni-bridge/render.h`	Minimal stub for `renderprefs.h` include chain.
`jni-bridge/renderprefs.h`	Minimal stub — `play.c` includes it but uses no rendering symbols.
`jni-bridge/sound.h`	Not needed — real `engine-core/sound.h` exists and is used.
`engine-core/config.h`	Host-generated autotools config, patched to remove flags invalid on Android.
`engine-core/lib/config.h`	Single-line wrapper: `#include "../config.h"`.
`engine-core/lib/neuralnetsse.c`	Modified: VLA replaced with `posix_memalign`. See §5.6.
`jni-bridge/src/stubs.c`	Remaining GTK/UI stubs, TLD init, rollout infrastructure. See §5.4, §5.5, §5.7.
`jni-bridge/src/android-app.c`	**Android application shell** replacing `gnubg.c`. See §5.8.
`jni-bridge/src/native-lib.c`	JNI entry points for all Engine methods.
`jni-bridge/src/com/clavierhaus/gnubg/Engine.kt`	Kotlin object declaring all external (JNI) functions.

1.5.2 5.2 Engine Source Files in Build

The following `engine-core` source files are compiled into `libgnubg-engine.so`:

File	Role
`eval.c`	Evaluation engine core, neural network inference, move generation
`dice.c`	Dice RNG (SFMT/Mersenne Twister; GMP/BBS and random.org disabled)
`bearoff.c`, `bearoffgammon.c`	Bearoff database
`positionid.c`	Position ID encoding/decoding
`matchequity.c`, `mec.c`	Match equity tables
`util.c`, `file.c`, `boardpos.c`	Utility functions
`multithread.c`	Threading infrastructure (single-threaded on Android)

File	Role
`rollout.c`	Monte Carlo rollout engine
`sgf.c`	SGF file handling
`sgf_y.c`	SGF parser (bison-generated from `sgf_y.y`)
`sgf_l.c`	SGF lexer (flex-generated from `sgf_l.l`)
`play.c`	Game state machine, match management, move records
`analysis.c`	Position analysis, skill classification
`format.c`	Move and equity formatting
`formatgs.c`	Game statistics formatting
`drawboard.c`	Board ASCII rendering and move formatting (`FormatMove`)
`external.c`	External player protocol (POSIX sockets)
`glib-ext.c`	GLib extension utilities (GValue/GObject helpers)
`matchid.c`	Match ID encoding/decoding
`set.c`	Settings commands and configuration
`renderprefs.c`	Render preferences (GTK sections guarded)
`lib/output.c`	Output formatting
`lib/SFMT.c`	SIMD-oriented Fast Mersenne Twister
`lib/neuralnet.c`	Neural network evaluation
`lib/neuralnetsse.c`	NEON neural net implementation (patched)
`lib/cache.c`, `lib/md5.c`, `lib/isaac.c`, `lib/list.c`	Supporting data structures
`lib/inputs.c`	Neural net input feature computation

1.5.3 5.3 The config.h Shadow Mechanism

This is the most architecturally significant decision in the JNI bridge build.

The upstream `engine-core/config.h` is generated by GNU autotools `./configure` on the Fedora host. It contains `#define` flags for features present on Fedora that do not exist on Android:

```
#define HAVE_LIBGMP 1            // GMP (dice.c BBS RNG)
#define USE_SIMD_INSTRUCTIONS 1  // x86 SSE/AVX — invalid on aarch64
#define HAVE_SSE 1
#define USE_SSE2 1
#define USE_AVX 1
#define LIBCURL_PROTOCOL_HTTPS 1 // randomorg.c
#define HAVE_LIBCURL 1
#define USE_MULTITHREAD 1        // GLib-based thread pool
```

These flags cannot be overridden via `-U` on the compiler command line because the C standard specifies that `#include "file.h"` (quoted include) searches the source file's own directory before any `-I` paths. This means `eval.c`'s `#include "config.h"` resolves to `engine-core/config.h` regardless of any `-I` flags pointing elsewhere.

The solution — the shadow file mechanism: a file named `config.h` is placed in each directory from which source files perform a quoted `config.h` include. Each shadow file includes the real `config.h` via a relative path, undefines the invalid flags, and adds Android-specific defines.

Two shadow files are required:

- `jni-bridge/config.h` — intercepts includes from `engine-core/*.c`
- `engine-core/lib/config.h` — intercepts includes from `engine-core/lib/*.c`

The current shadow config adds NEON defines after stripping x86 SIMD:

```
#include "../engine-core/config.h"
#undef HAVE_LIBGMP
#undef LIBCURL_PROTOCOL_HTTPS
#undef HAVE_LIBCURL
#undef USE_SIMD_INSTRUCTIONS   // strip x86 SIMD first
#undef HAVE_SSE
#undef USE_SSE2
#undef USE_AVX
// ARM NEON — mandatory on aarch64, replaces x86 SIMD:
#define USE_SIMD_INSTRUCTIONS 1
#define HAVE_NEON 1
#define USE_NEON 1
```

The real `engine-core/config.h` is also regenerated without the invalid flags using `grep -v`, making the shadow undefines redundant but retained as defense-in-depth. **This approach requires no modification to any upstream source file** (except `neuralnetsse.c`; see §5.6).

1.5.4 5.4 stubs.c Design

5.4.1 Global Variable Storage Several gnubg globals are declared `extern` in headers but their storage is defined in desktop-layer source files not in the Android build. `stubs.c` provides the storage with correct types:

- `matchstate ms` — the global match state struct
- `player ap[2]` — the two player structs
- `int positions[2][30][3]` — board position geometry array (from `boardpos.h`)
- `ThreadData td` — the thread pool state struct (zero-initialised at declaration; populated by `gnubg_init_tld()`)
- `const char *szHomeDirectory`, `char *szCurrentFileName` — path globals

5.4.2 Function Stubs Functions belonging to the GTK/UI/desktop layer are stubbed with signatures matching the header declarations exactly:

- **Threading:** `Mutex_Lock`, `Mutex_Release`, `ResetManualEvent`, `SetManualEvent`, `WaitForManualEvent`, `InitManualEvent`, `FreeManualEvent`, `InitMutex`, `CloseThread`, `TLSSetValue`, `MT_CreateThreadLocalData`
- **Locking wrappers (EXP_LOCK_FUN):** `EvaluatePositionWithLocking`, `FindBestMoveWithLocking`, `FindnSaveBestMovesWithLocking`, `GeneralCubeDecisionEWithLocking`, `GeneralEvaluationEWithLocking`, `ScoreMoveWithLocking`, `BasicCubefulRolloutWithLocking` — each delegates to the corresponding `NoLocking` variant (single-threaded evaluation)
- **UI/progress:** `ProcessEvents`, `ProgressValue`, `ProgressStart`, `ProgressEnd`, `ProgressValueAdd`
- **Callbacks:** `LogCube`, `SetRNG`, `ChangeGame`, `FormatMove`, `get_current_moverecord`
- **Network dice:** `RandomorgDice`, `NetworkDice` (returns -1; requires libcurl)
- **Match state:** `save_autosave`, `msBoard`

5.4.3 Disabled Features

Feature	Reason disabled	Re-enablement
GMP/BBS RNG (`dice.c`)	Requires `libgmp`, unavailable on Android	Add libgmp cross-build; restore `HAVE_LIBGMP` in `config.h`
x86 SIMD	`HAVE_SSE`, `USE_SSE2`, `USE_AVX` invalid on aarch64	N/A — replaced by NEON
GNU Multithread pool	Previously disabled; `USE_MULTITHREAD` now active	Re-enabled in Phase 9 via `GThreadPool/GPrivate`
libcurl / random.org	`LIBCURL_PROTOCOL_HTTPS` / `HAVE_LIBCURL` removed; `randomorg.c` excluded	Cross-compile libcurl for Android; restore flags and add `randomorg.c` to build

1.5.5 5.5 Thread-Local Data Initialisation (Critical)

Background: When `USE_MULTITHREAD` is **enabled** (Phase 9+), gnubg's move generation and scoring macros expand as follows:

```
#define MT_Get_aMoves()   ((ThreadLocalData *)TLSGet(td.tlsItem))->aMoves
#define MT_Get_nnState()  ((ThreadLocalData *)TLSGet(td.tlsItem))->pnnState
```

Both call `TLSGet()` which is implemented via `GPrivate` — each thread gets its own `ThreadLocalData` allocated lazily on first access. This replaces the previous single-threaded approach of a single static `gnubg_tld` struct.

```
#define MT_Get_aMoves()   td.tld->aMoves
#define MT_Get_nnState()  td.tld->pnnState
```

Both dereference `td.tld`, a `ThreadLocalData *` inside the global `ThreadData td`. At program start, `td` is zero-initialised, making `td.tld = NULL`. The first call to 1-ply evaluation triggers `GenerateMoves` → `MT_Get_aMoves()` → null dereference at offset 0x8. Similarly, `ScoreMoves` calls `MT_Get_nnState()` → `td.tld->pnnState`, also null, causing a write through null at offset 0x40 into the NNState array.

The fix — `gnubg_init_tld()`: A function in `stubs.c` that must be called once after `EvalInitialise()` and before any evaluation:

```
static ThreadLocalData gnubg_tld;
static NNState         gnubg_nn_states[3];
static move            gnubg_moves[MAX_MOVES];

void gnubg_init_tld(void) {
    unsigned int i;

    gnubg_tld.aMoves    = gnubg_moves;
    gnubg_tld.pnnState  = gnubg_nn_states;
    td.tld              = &gnubg_tld;

    /* Allocate savedBase (cHidden floats) and savedIBase (cInput floats)
     * for each NNState entry. Sizes are read from nnContact after weight load.
     * Three entries cover CLASS_RACE, CLASS_CRASHED, CLASS_CONTACT offsets. */
    for (i = 0; i < 3; i++) {
        gnubg_nn_states[i].state      = NNSTATE_NONE;
        gnubg_nn_states[i].savedBase  = g_malloc(nnContact.cHidden * sizeof(float));
        gnubg_nn_states[i].savedIBase = g_malloc(nnContact.cInput  * sizeof(float));
    }
}
```

Why 3 NNState entries: `ScoreMoves` accesses `nnStates[0..2]` directly, and `EvaluatePositionFull` indexes by `pc - CLASS_RACE` where `pc` ranges from `CLASS_RACE` (8) to `CLASS_CONTACT` (10), giving offsets 0, 1, 2.

Call site in native-lib.c: `gnubg_init_tld()` must be called inside `Java_com_clavierhaus_gnubg_Engine_initia` after `EvalInitialise()` returns, while `gnubg_lock` is held.

Call site in test harness: Called immediately after `EvalInitialise()` in `main()`.

1.5.6 5.6 NeuralNetEvaluateSSE VLA Alignment Patch

Background: `NeuralNetEvaluateSSE` in `neuralnetsse.c` declares its hidden-layer activation buffer as a Variable Length Array with an alignment attribute:

```
SSE_ALIGN(float ar[pnn->cHidden]);
// expands to: float ar[pnn->cHidden] __attribute__((aligned(16)));
```

On aarch64 with Clang 18, the `__attribute__((aligned(16)))` on a VLA is not guaranteed to be honoured. The NEON intrinsics (`vld1q_f32`, `vst1q_f32`) then operate on a potentially misaligned buffer, causing a SIGSEGV.

The fix: Replace the VLA with a heap-allocated aligned buffer:

```
float *ar = NULL;
if (posix_memalign((void **)&ar, ALIGN_SIZE, pnn->cHidden * sizeof(float)) != 0)
    return -1;
EvaluateSSE(pnn, arInput, ar, arOutput);
free(ar);
return 0;
```

`posix_memalign` guarantees `ALIGN_SIZE` (16 bytes for NEON) alignment. This is the only modification made to a file in `engine-core/lib/` beyond the shadow `config.h`. It is documented in `PROVENANCE.md`.

1.5.7 5.7 Rollout Infrastructure in stubs.c

5.7.1 Evolution Phase 7 introduced a synchronous single-threaded rollout bypassing `MT_WaitForTasks`. Phase 9 replaced this with a full multi-threaded implementation using `GThreadPool`. Both share the same external API (`gnubg_rollout()`) and the same `gnubg_init_rollout()` initialisation call.

5.7.2 Multi-threaded Architecture (Phase 9) `gnubg_rollout()` orchestrates a parallel rollout using a persistent `GThreadPool`:

Initialisation (`gnubg_init_rollout()`): - Allocates `rngctxRollout` by copying the main RNG context via `CopyRNGContext` - Creates a persistent `GThreadPool` with `g_get_num_processors()` threads - Pool is reused across all rollout calls to avoid thread creation overhead

Thread-local state (two `GPrivate` keys): - `gnubg_tls_key` — owns `ThreadLocalData` per thread: `aMoves[MAX_MOVES]`, `pnnState[3]` with `savedBase`/`savedIBase` buffers sized from `nnContact` dimensions. Destructor frees all buffers. - `gnubg_rng_key` — owns an isolated `rngcontext` copy per thread. Copied from `rngctxRollout` on first use. Destructor calls `free_rngctx()`. This is the critical RNG isolation that prevents data races.

Scatter-gather rollout loop: 1. Allocate `nTrials` result structs with 64-byte cache-line alignment (`posix_memalign`) to prevent false sharing 2. Initialise `GMutex`/`GCond` barrier with `tasks_remaining = nTrials` 3. Push `nTrials` tasks to pool via `g_thread_pool_push()` 4. Block on `g_cond_wait()` until all workers signal completion

Per-trial worker (`rollout_worker_func`): - Retrieves/allocates thread-local `ThreadLocalData` via `TLSGet()` - Retrieves/allocates thread-local `rngcontext` via `gnubg_rng_key` - Creates per-task `perArray` with seed offset by `task_index` for quasi-random dice - Calls `BasicCubefulRolloutNoLocking()` with the full 13-argument signature - Writes result to its designated slot in the scatter array - Decrements `tasks_remaining` and signals `GCond` when it reaches zero

Scatter-gather merge (main thread after barrier): - Accumulates sum and sum-of-squares over all trial results - Computes mean and standard deviation per output

5.7.3 Rollout globals The following globals are defined in `stubs.c` with defaults matching gnubg's standard money-game settings. These are the "docking points" between the engine and the UI layer:

- `rolloutcontext rcRollout` — default rollout configuration (1296 trials, cubeful, variance reduction, quasi-random dice, Mersenne Twister RNG)
- `rngcontext *rngctxRollout` — RNG context for rollouts; allocated by `gnubg_init_rollout()` via `CopyRNGContext(rngctxCurrent)`
- `int fAutoCrawford`, `fAutoSaveRollout`, `fShowProgress` — match/UI state flags
- `int fOutputMWC`, `fOutputWinPC`, `fOutputMatchPC` — output format flags (declared in `format.h` which is stubbed)

5.7.2 gnubg_init_rollout() Must be called after `EvalInitialise()` and `gnubg_init_tld()`. Allocates `rngctxRollout` by copying the current RNG context established during engine initialisation. `rngcontext` is an opaque type defined privately in `dice.c`; `CopyRNGContext` is the only correct way to create an instance from outside that translation unit.

```
int gnubg_rollout(const TanBoard anBoard,
                  float arOutput[NUM_ROLLOUT_OUTPUTS],
                  float arStdDev[NUM_ROLLOUT_OUTPUTS],
                  const cubeinfo *pci, rolloutcontext *prc);
```

5.7.3 gnubg_rollout() Calls `BasicCubefulRolloutNoLocking` for each of `prc->nTrials` games. Uses `QuasiRandomSeed()` (copied from `rollout.c` where it is `static`) to initialise the quasi-random dice permutation array. Accumulates per-game outputs and computes mean and standard deviation on completion.

5.7.4 QuasiRandomSeed **Note on `QuasiRandomSeed`:** This function is `static` in `rollout.c` with no header declaration. It cannot be linked from outside. The function body was copied verbatim into `stubs.c`; it uses only `irandinit`/`irand` from `lib/isaac.c` which is already in the build. The copy is documented in `PROVENANCE.md`.

5.7.5 Rollout accuracy With `nTrials=144` (the default minimum) the standard deviation on win probability is approximately ± 0.005. For publication-quality results use `nTrials=1296` or higher. The opening position rollout equity (0.29 at 144 trials) diverges from the 1-ply neural net estimate (0.077) due to sampling variance; convergence improves with more trials.

1.5.8 5.8 android-app.c — The Android Application Shell

`jni-bridge/src/android-app.c` is the architectural centrepiece of Phase 8. It replaces `gnubg.c` (the GTK application shell, ~8000 lines) on Android.

5.8.1 Design Principle On desktop gnubg, `gnubg.c` serves two roles: 1. **Portable application logic** — `InitBoard`, `NextToken`, `ParseNumber`, `find_skills`, `GetMatchStateCubeInfo` etc.

— pure C with no GTK dependency 2. **GTK application shell** — `main()`, GTK initialisation, window management, signal handlers, readline, Python module, curl

On Android, role 1 is extracted verbatim into `android-app.c`. Role 2 is replaced by Android callbacks that post events to the Kotlin layer. `progress.c` (GTK rollout progress dialog) and `sound.c` are also replaced here.

5.8.2 Android Callback Architecture Six weak-symbol callbacks form the interface between the C engine and the Android UI layer:

```
void gnubg_on_sound_event(gnubgsound gs);
void gnubg_on_setting_changed(void *pv);
void gnubg_on_board_changed(void);
void gnubg_on_navigation_event(int ev);
void gnubg_on_filename_changed(const char *sz);
void gnubg_on_rollout_progress(int iGame, int nTrials, float rJsd, int fStopped);
```

Declared `__attribute__((weak))` with no-op defaults. `native-lib.c` overrides them with implementations that call back into the JVM.

5.8.3 Stub Policy The principle: **stub only what is genuinely GTK rendering code.** Everything else is real implementation.

Function	Status	Future
`playSound`	Routes to `gnubg_on_sound_event`	Android SoundPool
`ShowBoard`	Routes to `gnubg_on_board_changed`	Android board view
`UpdateSetting`	Routes to `gnubg_on_setting_changed`	Android settings binding
`get_input_discard`	Returns TRUE	Android confirmation dialog
`confirmOverwrite`	Returns TRUE	Android file dialog
`GiveAdvice`	Routes to navigation callback	Android tutor dialog
`GetInputYN`	Returns TRUE	Android yes/no dialog
`CommandFirstGame/Move`	Routes to navigation callback	Android navigation
`RolloutProgressStart/ProgressEnd`	Routes to rollout progress callback	Android progress UI
`HandleCommand`	No-op	Android command layer
`hint_double/take/move`	No-op	Android tutor hints
`ExtInitParse/ExtStartParse/ExtDestroyParse`	Stubs	External player protocol
`SetupLanguage`	Returns NULL	Android locale system

1.6 6. JNI API Reference

1.6.1 6.1 Board Encoding

The board is passed as a `jintArray` of 50 elements encoding `TanBoard anBoard[2][25]`:

- Elements 0–24: `anBoard[0][i]` — opponent's checker counts
- Elements 25–49: `anBoard[1][i]` — current player's checker counts

1.6.2 6.2 Kotlin API (Engine object)

Method	Description
`initialise(weightsPath: String): Boolean`	Must be called once before any evaluation. Pass the absolute path to `gnubg.weights` extracted to internal storage. Calls `EvalInitialise()`, `SetCubeInfo()`, `gnubg_init_tld()`, and `gnubg_init_rollout()`.
`evaluatePosition(board: IntArray): FloatArray?`	Returns `FloatArray[5]`: `[winNormal, winGammon, winBackgammon, loseGammon, loseBackgammon]`. Null on engine error. Uses 1-ply cubeless `evalcontext`.
`findBestMove(board: IntArray, die0: Int, die1: Int): IntArray?`	Returns `IntArray[8]` encoding `anMove[8]`; unused slots are -1. Null on error.
`classifyPosition(board: IntArray): Int`	Returns the `positionclass` integer (race, contact, bearoff etc). Returns -1 if not initialised.
`applyMove(board: IntArray, move: IntArray): IntArray`	Applies a move to a board. Returns the resulting 50-element board encoding.
`cubeDecision(board, cubeValue, cubeOwner, matchTo, score0, score1, crawford): IntArray?`	Returns `IntArray[16]`: `[0..6]` = if-double equity floats as `Int` bits (unpack with `Float.fromBits()`), `[7..13]` = no-double equity floats, `[14]` = `CubeDecision` enum value, `[15]` = reserved. See `Engine.CubeDecision` enum for all 21 values.
`rollout(board: IntArray, trials: Int = 144): FloatArray?`	Multi-threaded cubeful rollout via `GThreadPool`. Returns `FloatArray[14]`: `[0..6]` = equity outputs, `[7..13]` = standard deviations. Use `trials=1296` or higher for publication-quality results.
`loadSGF(path: String): Boolean`	Load a match from an SGF file. Calls `CommandLoadMatch()` which handles `FreeMatch`, `ClearMatch`, `RestoreGame`, `UpdateSettings`. Returns true on success.
`saveSGF(path: String): Boolean`	Save the current match to an SGF file. Calls `CommandSaveMatch()` which handles `SaveGame` loop and file I/O. Returns false if no game in progress.

1.6.3 6.3 Thread Safety

All JNI entry points acquire a static `pthread_mutex_t` (`gnubg_lock`) before calling any engine function and release it on return. The gnubg evaluation engine uses global state (NNState, bearoff database handles, weight arrays) that is not safe to access concurrently.

1.6.4 6.4 Initialisation & Data Files

The engine requires the following files at runtime, extracted from app assets to internal storage before calling `Engine.initialise()`:

- `gnubg.weights` — trained neural network weights (~6 MB). Primary source of playing strength.
- `gnubg_os0.bd` — small one-sided bearoff database (~1.4 MB, 6pt/15ch). Loaded into `pbc1`.
- `gnubg_ts0.bd` — small two-sided bearoff database (~6.8 MB, 6pt/6ch). Loaded into `pbc2`.
- `gnubg_os.bd` — larger one-sided bearoff database (~4.8 MB, 7pt/15ch). Loaded into `pbcOS`. Generated via `makebearoff -o 7 -O gnubg_os0.bd -f gnubg_os.bd`.
- `gnubg_ts.bd` — two-sided bearoff database (~6.6 MB, 6pt/6ch copy). Loaded into `pbcTS`. Copy of `gnubg_ts0.bd`.

All four databases are deployed and confirmed loading on device (`pbc1`, `pbc2`, `pbcOS`, `pbcTS` all non-null). The `AC_PKGDATADIR` path is compiled in as `/data/data/com.clavierhaus.gnubg/files` and must match the actual extraction location.

1.7 7. Build Instructions

1.7.1 7.1 Prerequisites (Fedora 44)

All prerequisites are available via dnf. No pip, npm, or external package managers are required:

```
sudo dnf install meson ninja-build cmake glib2-devel
```

NDK location: `/home/erweitert/gnubg-android/android-sdk/ndk/27.0.11718014/`

1.7.2 7.2 Build GLib for Android (one-time, ~3–5 minutes)

```
cd /home/erweitert/gnubg-android
./build_glib_android.sh 2>&1 | tee glib-build.log
```

On success: `jni-bridge/external/glib/lib/libglib-2.0.so` is created (ELF 64-bit ARM aarch64, Android 28).

1.7.3 7.3 Build libgnubg-engine.so

```
cd /home/erweitert/gnubg-android/jni-bridge
rm -rf build && mkdir build && cd build
cmake .. \
  -DCMAKE_TOOLCHAIN_FILE=
  ↪ /home/erweitert/gnubg-android/android-sdk/ndk/27.0.11718014/build/cmake/android.toolchain.cmake
  ↪ \
  -DANDROID_ABI=arm64-v8a \
  -DANDROID_PLATFORM=android-28 \
  -DCMAKE_BUILD_TYPE=Release
cmake --build . 2>&1 | tee /home/erweitert/gnubg-android/jni-build.log
```

1.7.4 7.4 Verification

```
file jni-bridge/build/libgnubg-engine.so
# Expected: ELF 64-bit LSB shared object, ARM aarch64, for Android 28

nm jni-bridge/build/libgnubg-engine.so | grep "T Java_"
# Expected (9 JNI entry points):
#   T Java_com_clavierhaus_gnubg_Engine_applyMove
#   T Java_com_clavierhaus_gnubg_Engine_classifyPosition
#   T Java_com_clavierhaus_gnubg_Engine_cubeDecision
#   T Java_com_clavierhaus_gnubg_Engine_evaluatePosition
#   T Java_com_clavierhaus_gnubg_Engine_findBestMove
#   T Java_com_clavierhaus_gnubg_Engine_initialise
#   T Java_com_clavierhaus_gnubg_Engine_loadSGF
#   T Java_com_clavierhaus_gnubg_Engine_rollout
#   T Java_com_clavierhaus_gnubg_Engine_saveSGF

nm jni-bridge/build/libgnubg-engine.so | grep "NeuralNetEvaluateSSE"
# Expected: T NeuralNetEvaluateSSE  (confirms NEON path active)

nm jni-bridge/build/libgnubg-engine.so | grep "gnubg_rollout\|BasicCubefulRollout"
# Expected: T gnubg_rollout, T BasicCubefulRolloutNoLocking
```

1.7.5 7.5 Android adb Test Harness

The test harness in `test-harness/` builds as a standalone Android executable and exercises the engine end-to-end on a connected device.

```
# Build
cd /home/erweitert/gnubg-android/test-harness
rm -rf build && mkdir build && cd build
cmake .. \
  -DCMAKE_TOOLCHAIN_FILE=
  ↪  /home/erweitert/gnubg-android/android-sdk/ndk/27.0.11718014/build/cmake/android.toolchain.cmake
  ↪  \
  -DANDROID_ABI=arm64-v8a \
  -DANDROID_PLATFORM=android-28 \
  -DCMAKE_BUILD_TYPE=Release
cmake --build .

# Push to device
adb push test_runner /data/local/tmp/gnubg-test_runner
adb push /path/to/gnubg.weights   /data/local/tmp/gnubg.weights
adb push /path/to/gnubg_os0.bd   /data/local/tmp/gnubg_os0.bd
adb push /path/to/gnubg_ts0.bd   /data/local/tmp/gnubg_ts0.bd
adb push jni-bridge/external/glib/lib/libglib-2.0.so /data/local/tmp/
adb push jni-bridge/external/glib/lib/libintl.so     /data/local/tmp/
adb shell chmod +x /data/local/tmp/gnubg-test_runner

# Run
adb shell "LD_LIBRARY_PATH=/data/local/tmp \
    /data/local/tmp/gnubg-test_runner /data/local/tmp/gnubg.weights"
```

Expected output:

```
=== GNU Backgammon Android Test Harness ===
[1] Initialising engine...
    pbc1=0x... pbc2=0x... pbcOS=0x... pbcTS=0x...  (all non-null)
[2] ClassifyPosition = 10 (expected 10 = CLASS_CONTACT)
[3] EvaluatePosition 1-ply cubeless...
    Win prob:        0.5269
    Win gammon:      0.1478
    ...
```

```
    Cubeless equity: 0.0768
[4] FindBestMove for 3-1...
    Move: 8/5 6/5 (expected 8/5 6/5)
[5] Cube decision (money game, centred cube)...
    Cube decision: 2 (NODOUBLE_TAKE)
    No double equity:   0.4260
    Double/take equity: 0.4260
    Double/pass equity: 0.2950
[6] Rollout (144 trials, cubeful variance reduction)...
    gnubg_rollout returned 0
    Win prob:  ~0.59  stddev: ~0.005
    Equity:    ~0.29  (converges toward 1-ply with more trials)
=== All tests passed ===
```

1.8 8. Open Items & Roadmap

1.8.1 8.1 Features

The engine is feature-complete as of Phase 9.

- **Rollout:** [DONE] Multi-threaded via `GThreadPool`/`GPrivate`. `Engine.rollout()`. See §5.7.
- **Cube decisions:** [DONE] `Engine.cubeDecision()`. Full equity matrix + `CubeDecision` enum (21 values).
- **SGF import/export:** [DONE] `Engine.loadSGF()` / `Engine.saveSGF()`. Backed by `CommandLoadMatch/Com` in `sgf.c`.
- **Game management:** [DONE] `play.c` compiled. Full match state machine.
- **Analysis:** [DONE] `analysis.c` compiled. `find_skills`, `AnalyzeMove`, `AnalyzeGame`, `AnalyzeMatch` available.
- **Multi-threading:** [DONE] `USE_MULTITHREAD` enabled. `GThreadPool` + `GPrivate` TLS + RNG isolation. See §5.7.
- **SQLite:** Not planned — use Android's Room/SQLiteOpenHelper for persistence.

1.8.2 8.2 Immediate Next Steps

- **Android Studio project:** Create the Android app project, add `libgnubg-engine.so` and GLib `.so` files as prebuilt native libraries, add `Engine.kt` to the source set.
- **Asset packaging:** Add `gnubg.weights` and all four bearoff databases to the app's assets. Implement extraction to `context.filesDir` on first launch.
- **Callback wiring:** Implement non-weak versions of `gnubg_on_*` callbacks in `native-lib.c` to route engine events into the Kotlin UI layer.

1.8.3 8.3 Performance

- **ARM NEON SIMD:** Enabled. `NeuralNetEvaluateSSE` confirmed exported and active. `CheckNEON()` always returns 1 on aarch64.
- **Multi-threading:** [DONE] `USE_MULTITHREAD` enabled. Rollout tasks distributed across all CPU cores via `GThreadPool`. Per-thread `rngcontext` isolation via `GPrivate` guarantees correctness. See §5.7.

1.8.4 8.4 Known Warnings

- **`multithread.c`:** 6 instances of incompatible pointer types passing `pthread_mutex_t *` to `Mutex *`. `USE_MULTITHREAD` is now enabled but these coercions are in paths that are compiled but not reached on Android; harmless in practice.

- **CMake deprecation warnings** from the NDK toolchain file regarding `cmake_minimum_required VERSION < 3.10`. In the NDK's own toolchain file; cannot be fixed from the project.
- **Rollout equity variance at low trial counts:** With `nTrials=144` stddev on win prob is ~0.005. Use 1296+ trials for reliable equity estimates.

1.9 9. Key File Reference

1.9.1 build_glib_android.sh

Orchestrates the complete GLib cross-build: applies Fedora SRPM patches, patches `meson.build` for Android iconv-in-libc, runs `meson setup` with `android-arm64.cross`, builds with ninja, installs to `jni-bridge/external/glib/`. Safe to re-run.

1.9.2 android-arm64.cross

Meson cross-file for the GLib build. Specifies NDK 27 toolchain binaries (`aarch64-linux-android28-clang`), target machine (android/aarch64/little-endian), compile flags (`-DANDROID -fPIC -Os`), and platform size properties. API level 28 is encoded in the compiler binary name, not as a `-D` flag.

1.9.3 jni-bridge/CMakeLists.txt

NDK CMake build file for `libgnubg-engine.so`. Enumerates all compiled source files, sets include paths (`jni-bridge/` first for shadow `config.h` interception, then `engine-core/`, `engine-core/lib/`, GLib headers), defines `AC_DATADIR`/`AC_PKGDATADIR`/`AC_DOCDIR` for Android, and links against `libglib-2.0.so`, `libgobject-2.0.so`, `libintl.so`, `liblog`, and `libm`.

1.9.4 jni-bridge/src/stubs.c

Provides remaining GTK/UI stub storage, TLD infrastructure (now via `GPrivate`), multi-threaded rollout orchestration (`gnubg_rollout()` with `GThreadPool`), and rollout globals. Key functions: `gnubg_init_tld()`, `gnubg_init_rollout()`, `gnubg_rollout()`, `QuasiRandomSeed()`, `TLSGet()`. **Note:** `test-harness/src/stubs.c` is a copy of this file and must be kept in sync.

1.9.5 jni-bridge/src/android-app.c

The Android application shell replacing `gnubg.c`. Contains functions extracted verbatim from `gnubg.c` (see §5.8) plus Android callbacks for sound, board, settings, navigation, and rollout progress. All globals previously in `gnubg.c` are defined here.

1.9.6 engine-core/config.h

Host-generated autotools `config.h`, regenerated by `grep -v` to strip Android-incompatible flags. **Important:** if `engine-core/` is refreshed from upstream, this file must be regenerated and re-patched using the `grep -v` pipeline documented in §5.3.

1.9.7 engine-core/lib/neuralnetsse.c

Modified from upstream: `NeuralNetEvaluateSSE` function has its VLA replaced with `posix_memalign` allocation for correct 16-byte alignment on aarch64. See §5.6 and `PROVENANCE.md` for full documentation of this change.

1.10 10. Android UI — IDE Decision, Design Philosophy & Way Forward

1.10.1 10.1 Milestone: Engine Complete

As of Phase 9, `libgnubg-engine.so` is feature-complete. All 9 JNI entry points are implemented and verified on a Pixel 8 Pro (aarch64, Android 16). The engine milestone closes the native layer of the project. Version v0.9 marks the transition from engine development to Android UI development, targeting a 1.0 release.

1.10.2 10.2 IDE Decision: No IDE — Pure Terminal Workflow

After evaluating all options, the decision is to use a **pure terminal workflow** with no Android Studio or other IDE involved at any stage. The rationale:

- Android Studio requires a GUI session, significant RAM overhead, and account/sign-in infrastructure that adds friction without benefit for this project.
- The entire build is driven by `gradle assembleDebug` from the terminal — the same Gradle invocation Android Studio uses internally.
- All source files are plain Kotlin and are edited in any text editor. The project has no XML layouts (see §10.3).
- The keyboard-operator workflow (human pastes commands, observes output) is better served by explicit, auditable shell commands than by IDE wizards.

The project structure is a bare Gradle project (`gnubg-app/`) created entirely from the terminal. No `.idea/` directory, no Gradle wrapper magic, no IDE-generated boilerplate beyond what is explicitly documented here.

1.10.3 10.3 UI Toolkit: Jetpack Compose

The Android UI is built exclusively with **Jetpack Compose** — Android's modern declarative UI toolkit. XML layouts are explicitly excluded. Rationale:

- Compose maps directly to the terminal/text-editor workflow: every UI element is a Kotlin `@Composable` function. There are no parallel XML resource files to maintain.
- Material You dynamic colour (`dynamicColorScheme()`) is first-class in Compose. All colour tokens defined in §10.5 are available as `MaterialTheme.colorScheme.*` with no additional plumbing.
- The board is drawn via Compose's `Canvas` composable — the same drawing model as Cairo/SVG, accepting the boarddim.h geometry directly.
- Animation (checker movement, dice roll) is `ObjectAnimator` over Compose state — no separate animation framework needed.

1.10.4 10.4 Build Configuration

Parameter	Value
`minSdk`	31 (Android 12 — dynamic colour guaranteed on all supported devices)
`targetSdk`	35 (Android 15 — current Play Store requirement)
`compileSdk`	35
Kotlin	2.0.21
Compose BOM	2024.09.03 (ships Compose 1.7)
AGP	8.5.2
Gradle wrapper	8.7

`minSdk = 31` means `dynamicColorScheme()` is available on every supported device with no

API version guards. Play Store reach at Android 12+ is approximately 75% of active devices — acceptable for a serious backgammon application not targeting the mass casual market.

1.10.5 10.5 Design Philosophy

Orientation Landscape only. No portrait mode is available or planned. `android:screenOrientation="lands` is set in `AndroidManifest.xml`. This fixes the aspect ratio and eliminates all layout reflow complexity.

Board Geometry The board geometry is derived directly from `boarddim.h` in the gnubg source. The canonical coordinate system is a 108×82 unit grid (aspect ratio 1.317:1). All element positions — point triangles, bar, bearoff trays, checker stacks, cube, hinges — are computed from the `POINT_X()`, `TOP_POINT_Y`, `BOT_POINT_Y`, `CHEQUER_HEIGHT`, and `CHEQUER_WIDTH` macros already present in the source. This is a one-time translation; the geometry is gnubg's own, faithfully reproduced.

Material You Colour System The board colour system is built entirely on Android's Material You dynamic colour API (`DynamicColors`, `dynamicColorScheme`). The device wallpaper seed generates the full tonal palette automatically. Three user-selectable seed overrides are offered:

Name	Seed	Character
Dynamic	Device wallpaper	Default — fully personalised
Classic	`#5D4037`	Warm brown → amber/ochre tones
Ocean	`#1565C0`	Deep blue (BG Galaxy territory)

Colour role assignments for board elements:

Element	Material You Role
Board field	`tertiaryContainer`
Triangle A (dominant)	`tertiary`
Triangle B (recessive)	`tertiaryContainer` at 60%
Bar	`surface` darkened
Border/frame	`secondaryContainer`
Point numbers	`onSecondaryContainer`
Dark checker fill	`primary` at P-20
Light checker fill	`primary` at P-90
Checker highlight	white at 25–30% opacity, offset top-left
Cube fill	`secondaryContainer`
Cube number	`onSecondaryContainer`
Die fill	`secondary`
Die pips	`onSecondary`

Rationale for tertiary board / primary checkers: Using `tertiary` for the board field and `primary` for the checkers guarantees perceptual distinctness across all wallpaper seeds, because Material You's algorithm explicitly assigns Primary and Tertiary to different hue regions. Both checker colours (P-20 and P-90) are within the same Primary hue family, giving a coherent monochromatic checker aesthetic while remaining legible against the Tertiary board.

Depth Treatment Checkers, cube, and dice share a consistent, minimal depth language:

1. **Base shape** — circle (checkers) or rounded-rect (cube/dice), solid fill.
2. **Rim** — 1.5px stroke, one tonal step darker than fill.
3. **Highlight ellipse** — white at 25–30% opacity, positioned at `(cx - 0.28r, cy - 0.28r)`, sized at `0.76r × 0.38r`. Simulates a diffuse overhead-left light source.

No drop shadows, no gradients, no bevels. The result reads as material without crossing into skeuomorphic excess.

Point Numbers Point numbers are placed **outside** the board border, in the frame area above and below the playing surface. They do not appear on the playing surface. This costs less than 5% of screen width and eliminates distraction during play. Players who do not use point numbers are unaffected.

UI Layer Architecture The Compose UI separates into two layers:

- **Static board layer** — a single `Canvas` composable drawing the board field, triangles, bar, bearoff trays, border, and point numbers. Drawn once; does not change during a game.
- **Dynamic piece layer** — checkers, dice, cube, legal-move highlights, and last-move trail. Positioned programmatically from `ChequerPosition()` and `CubePosition()` in `boardpos.c` via JNI. Checker movement is an `ObjectAnimator` over Compose state.

This separation means animations touch only the dynamic layer; the board surface is never redrawn during play.

Functionality Philosophy The UI presents a clean, modern surface while exposing 100% of gnubg's analytical depth. Two access modes:

- **Standard mode** — board, dice, cube controls, score display. Clean and uncluttered.
- **Expert/Advanced mode** — analysis panel showing equity, error classification, rollout button, hint mode, tutor mode, SGF import/export. Everything the desktop version offers, surfaced progressively.

No gnubg functionality is removed or stubbed at the UI level. The distinction is presentation only.

1.10.6 10.6 Immediate Next Steps

1. Create the bare Gradle project structure (`gnubg-app/`) from the terminal — `settings.gradle.kts`, `build.gradle.kts`, `gradle/libs.versions.toml`, `AndroidManifest.xml`, `MainActivity.kt`, `ui/theme/Theme.kt`.
2. Verify `gradle assembleDebug` produces a clean APK with an empty Compose screen.
3. Implement `Board.kt` — the static board `Canvas` composable from boarddim.h geometry.
4. Implement `BoardState.kt` and `EngineViewModel.kt` — the bridge between the JNI engine and Compose state.
5. Wire checkers, dice, cube as the dynamic piece layer.

6. Add the analysis panel as the Expert mode overlay.

1.11 11. Version v0.9.1 — Basic UI Complete

1.11.1 11.1 Milestone

Version v0.9.1 marks the completion of the basic Android UI layer. The app is fully functional as a visual shell: the board renders correctly on any screen size and aspect ratio, all 24 points are geometrically accurate, checkers are placed in the starting position, dice and cube are rendered, and the settings system is fully wired to a `GameViewModel`. The next phase is engine integration — wiring the JNI bridge to live gameplay.

1.11.2 11.2 UI Components Delivered

Board (`Board.kt`)

- Geometry derived directly from `boarddim.h` — 24 points, asymmetric layout with single right-side bearoff tray, narrower centre bar
- Ocean blue fixed palette (foundation for Material You theme system)
- Point numbers in frame border (toggleable)
- Checkers in starting position — size governed by triangle height, 5 checkers = triangle height
- Side-on rectangle representation for bearoff tray checkers (15 per tray)
- Placeholder dice (3–1) with pip faces, correct player placement
- Doubling cube (64) centred in bar
- Pip counts in bar, equidistant from frame edges (toggleable)
- Two distinct bearoff trays (top/bottom) with white inner outline and dark outer border

Layout (`GameLayout.kt`)

- Landscape-only, fully immersive (status bar and nav buttons hidden)
- Screen keep-on during gameplay
- 25% player panel placeholder / 75% board split
- Settings gear icon (Material Icons) top-left, safe from screen rounded corners
- Full-screen settings overlay on tap

Settings System

- `GameViewModel` (`engine/GameViewModel.kt`) — single source of truth for all settings and future game state, using `StateFlow`
- `GameSettings` data class with 16 typed fields covering all four settings categories
- `SettingsScreen.kt` — full-screen BMPCC-inspired settings UI: horizontal tab bar (GAME / BOARD / ENGINE / ANALYSIS), scrollable sections, live-wired toggles and controls
- All switches toggle immediately and propagate to the board
- Match length and automatic doubles use +/-2/+/-1 steppers
- Difficulty selector (Beginner/Intermediate/Advanced/Expert) maps to gnubg ply depth
- Error classification thresholds displayed (editable in next phase)

Theme Infrastructure

- `BoardTheme` enum: OCEAN / CLASSIC / FOREST

- `Difficulty` enum with ply mapping
- Settings UI radio buttons wired to `GameViewModel` — colour theme switching renders in next phase

Build Environment

- SDKMAN manages Java 21.0.7-tem and Gradle 8.14 via `.sdkmanrc`
- `local.properties` points to `/home/erweitert/gnubg-android/android-sdk`
- `gradle.properties`: `android.useAndroidX=true`, `android.suppressUnsupportedCompileSdk=35`
- KDE Connect established as the screenshot-to-desktop workflow
- 16KB ELF alignment warning noted — to be resolved in engine rebuild phase with `-Wl,-z,max-page-size=16384`

1.11.3 11.3 Immediate Next Steps (v0.9.2)

1. Implement colour theme switching — OCEAN/CLASSIC/FOREST palettes derived from Material You or fixed grayscale
2. Wire engine initialisation — call `Engine.initialise()` with weights path on app start
3. Implement `BoardState.kt` — live board encoding from engine `matchstate`
4. Wire `Engine.findBestMove()` and `Engine.rollout()` to the game loop via `GameViewModel`
5. Implement touch input for checker moves — tap point to select, tap destination to move
6. Wire `Engine.cubeDecision()` to the doubling cube UI
7. Rebuild engine `.so` files with 16KB page alignment
8. Populate player panel (left 25%) with score, match state, timer

---

GNU Backgammon Android Port — MASTER v0.9.1 — clavierhaus.at — June 2026